

Agent Memory 专题 — HuggingFace 月榜 (2025-11 至 2026-06)

数据范围：HuggingFace Papers 月榜 2025-11、2025-12、2026-01、2026-02、2026-03、2026-04、2026-05、2026-06（共 8 个月，每月 top50）

筛选焦点：**Agent Memory**，尤其是 **working memory**（工作记忆）——即 **agent** 在多轮 / 长程任务中，随着 **transcript** / 上下文不断增长，如何表示、更新、使用这部分"当前状态"。兼收紧邻的 long-context 记忆管理、外化（externalization）、记忆 skill 化等工作。

分析三维度（贯穿全专题）：

① 怎么表示（**Representation**）：记忆以什么形式存在

growing transcript / 环境侧结构化状态（candidate pool、evidence links） / 文件系统 / 演进报告 / KV-cache / 固定大小关联状态矩阵 / 参数（adapter） / markdown skill。

② 怎么更新（**Update**）：记忆何时、由谁、按什么策略写入

与修订LLM 自己边推理边记 / 环境侧自动 bookkeeping / 周期性重构 / delta-rule 在线更新 / patch 历史 / RL 学到的 memorization policy / Sleep 阶段离线巩固。

③ 怎么使用（**Usage**）：记忆如何回到上下文供推理

budget-aware context rendering / 把 stale 观测 mask 掉 / 运行时 deep-research 式按需检索 / 低秩修正注意力 / 分阶段 rubric 召回。

long-horizon agent 的长上下文是一个需要主动管理的、不断增长的状态：

1. 已看过什么（检索回来的海量 observation）；
2. 哪些证据有用（evidence）；
3. 哪些约束还没满足（open constraints）；
4. 哪些断言已被核实（verified claims）。

朴素做法是把这些全堆进一个**单一、膨胀的 transcript**，让策略模型（policy）一边决定"下一步搜什么"，一边还要充当"记账员"记住上面这一切。多篇论文（[01-harness-1](#)、[2-iterresearch](#)（见-deep-research02））共同指出这会导致：

- **context suffocation**（上下文窒息）：有效信息被无关 token 稀释，推理容量被吃满；
- **noise contamination**（噪声污染）：早期失败路径、过时观测一直挂在上下文里干扰决策；

- **RL 目标错配**：强化学习被迫同时优化"语义搜索决策"和"本可由环境可靠维护的记账"，样本效率低、泛化差。

于是 **working memory** 成了独立研究对象：不是问"agent 长期记住了什么知识" (**long-term / episodic / factual memory**)，而是问"在当前这个长任务里，那团正在增长的状态该怎么表示、怎么更新、怎么用"。这正是与传统"agent memory / RAG"的区别——后者重 long-term 知识库，本专题重 运行时的 **working set**。

概念坐标 (forms × functions × dynamics 框架)：

- **功能维度**：factual (事实) / experiential (经验) / **working** (工作)
- **形式维度**：token-level / parametric / latent——working memory 主要落在 **token-level** (上下文里的结构化状态)，但 07-delta-mem 探索了 **latent** (固定状态矩阵)、06-qwenlong-l15 触及 **parametric**。
- **动态维度**：formed → evolved → retrieved

一、按"表示 / 更新 / 使用"三维度归类 (核心地图)

同一篇论文可能在三个维度都有贡献，下表按其**最具代表性的贡献维度**归位；完整三维拆解见各精读笔记的"表示/更新/使用"小节。

① 怎么表示 (Representation) ——记忆以什么形式承载

表示形式	代表论文	一句话
环境侧结构化工作记忆 (candidate pool + curated set + evidence links + verification records)	01-harness-1 ★	把"记账"从 policy 搬到 harness (环境侧)，policy 只留语义决策
演进报告 (evolving report) 作为记忆	IterResearch (见 deep-research/02)	每轮重建工作区，把洞见合成进一份不断演进的 report
文件系统 (hierarchical knowledge base) 作为外部记忆	03-fs-researcher ★	文件系统当持久工作区，知识库可远超 context length

表示形式	代表论文	一句话
持久树 (hypothesis tree) 链接假设/证据/洞见	hypothesis-tree-arbor 	长程科研中用一棵跨时间的树承载策略与证据
固定大小关联状态矩阵 (associative memory state)	07-delta-mem ★	8×8 在线状态矩阵压缩历史，耦合进注意力
KV-cache 作为分层记忆	hermes  / MSA 	把 KV-cache 重新概念化为多粒度记忆
结构化/可演化的 markdown skill 作为记忆	11-memskill-memento ★	记忆 = 可学习、可演化的"记忆技能"或 skill 文件
page-store + 轻量记忆 (JIT)	GAM (见 deep-research/11)	离线无损保管完整历史，运行时再现场编译

② 怎么更新 (Update) ——何时、由谁、按什么策略写入/修订

更新机制	代表论文	一句话
环境侧自动 bookkeeping (去重、压缩、重要性打标)	01-harness-1 ★	环境可靠维护可恢复状态，RL 不再学记账
周期性工作区重构 (Markovian state reconstruction)	IterResearch (deep-research/02)	把长程重构为 MDP，周期性合成→重置工作区
delta-rule 在线更新 (不动 backbone 权重)	07-delta-mem ★	用 delta 规则在线更新固定状态矩阵
patch 化更新历史 (记录环境演化)	08-evoarena-evomem ★	以结构化 update history 记录环境变化

更新机制	代表论文	一句话
RL 学到的 memorization policy (学"该记什么")	taskmem 	用 RL 让 agent 动态决定记什么
Sleep / Dreaming 离线巩固 (短期→长期参数)	10-externalization-efficiency 关联 / sleep 	睡眠阶段把脆弱短期记忆蒸馏进稳定长期参数
Read-Write 反思式学习 (边做边写 skill 库)	11-memskill-memento 	读阶段选 skill、写阶段更新/扩充 skill 库
Retrospective 自监督优化 (用过去轨迹改 harness)	rho 	无需标注, 自我偏好挑选 harness 更新

③ 怎么使用 (Usage) —— 记忆如何回到上下文供推理

使用方式	代表论文	一句话
budget-aware context rendering (按预算渲染上下文)	01-harness-1 	在 token 预算内把 curated 状态渲染回上下文
mask 掉 stale 观测 (token-for-turn 权衡)	02-masking 	把模型已不再 attend 的旧观测移出上下文
task-aware 自适应剪枝 (按当前目标 skim)	05-swe-pruner 	给定任务目标, 神经 skimmer 动态选相关行
运行时 deep-research 式按需检索	GAM (deep-research/11) / mia 	把"记忆检索"当成一次深度研究
单遍推理 ⇔ 迭代记忆处理融合	06-qwenlong-l15 	>4M token 时在单遍与记忆 agent 间无缝切换

使用方式	代表论文	一句话
低秩修正注意力（readout 改 attention）	07-delta-mem ★	记忆 readout 对注意力做低秩修正
把散落 turn 编译成显式长上下文监督	04-acc ★	把跨 turn 的证据编译成 long-context QA 训练信号
分阶段 rubric 召回	rubricem 📄	rubric 同时充当执行/评判/记忆的统一接口

一句话总览：2026 H1 的 working-memory 研究，表示上从"单一 transcript"转向"环境侧结构化状态 / 文件系统 / 固定状态矩阵"；更新上从"LLM 顺手记"转向"环境自动维护 / 周期重构 / 在线 delta / RL 学策略 / 离线巩固"；使用上从"全量塞回"转向"按预算渲染 / mask / 剪枝 / 按需检索 / 低秩注入"。三者合流指向同一句话——不要让 policy 既当大脑又当记账员，把可恢复的状态外化出去、把上下文当成需要主动调度的资源。

二、按类别列出（每篇 500–1000 字详述，摘要均基于 arXiv 原文）

下列按合理主题类别组织（同一篇若跨类，归入最贴切的一类）；每条保留月份/票数与 arXiv 编号。★ = 精读，📄 = 简短描述。每篇展开为 500–1000 字的详细摘要，覆盖问题动机 / 核心方法 / 关键结果 / 在"表示·更新·使用"三维度的定位 / 局限与意义。

A. 记忆即运行时重构（Memory as Runtime Reconstruction）

不在离线把记忆压死，而是运行时按需检索、整合、重建上下文。

General Agentic Memory Via Deep Research ★（已收录 deep-research/11） [BAAI, 171↑ | 2025-11] — arXiv 2511.18423

问题动机：AI agent 的记忆系统普遍采用 **static memory（静态记忆）** 范式——在交互过程中就提前把信息加工成"即取即用"的结构化记忆（如三元组、摘要、向量）。这种"预先消化"看似高效，却必然带来**严重的信息损失**：你在写入时并不知道未来会被问什么，任何提前压缩都会丢掉当时看似无关、日后却关键的细节。一旦丢失，下游再强的推理也无法把它找回。

核心方法：General Agentic Memory（GAM）借用编译器领域的 **JIT（Just-In-Time，即时编译）** 原则，提出一套 **duo-design（双角色设计）**：① **Memorizer（记忆器）**——在交互时只用一层轻量记忆

标注关键历史信息的"索引/高亮", 同时把**完整原始历史无损保存**在一个 universal page-store (通用页存储) 里, 绝不提前压缩丢弃; ② **Researcher (研究器)** ——在运行时收到在线请求后, 在预构建的轻量记忆引导下, **像做一次 deep research 一样**从 page-store 中按需检索、交叉验证、整合出当前问题真正需要的信息。换言之, "记忆检索"被重新定义为"一次以历史为语料的深度研究"。这一设计充分利用了 frontier LLM 的 agentic 能力与 test-time 可扩展性, 且整条 memorizer→researcher 流水线可端到端 RL 优化。

关键结果: GitHub 855★。在长程记忆基准上显著优于 static memory 基线, 尤其在"需要组合多个分散历史片段"的查询上优势明显——因为完整历史始终在 page-store 里没被丢掉。

三维定位: 表示=page-store (无损完整历史) + 轻量记忆索引; **更新**=离线只打高亮、不做有损压缩; **使用**=运行时现场"研究式"编译上下文。详见 deep-research/11-general-agentic-memory.md, 本专题作为"记忆 = 运行时重构"主线的**源头**交叉引用。

意义: GAM 把"记忆系统"从"提前建好的知识库"翻转为"运行时按需重建的研究过程", 是 working-memory 领域"don't compress early, reconstruct on demand"思想的奠基性表达, 直接启发了后续 MIA、Harness-1 等"环境侧保管完整状态 + 运行时 curate"的设计。

IterResearch: Rethinking Long-Horizon Agents via Markovian State Reconstruction ★ (已收录 deep-research/02) [80↑ | 2025-11] — arXiv 2511.07327

问题动机: 现有 deep-research agent 普遍采用 **mono-contextual paradigm (单一上下文范式)** ——把每一步检索到的所有信息无脑堆进同一个不断膨胀的上下文窗口。随着任务推进 (可达数百步), 这会引发两个致命问题: **context suffocation (上下文窒息)** ——有效信息被海量无关 token 稀释, 模型注意力被吃满、推理容量骤降; **noise contamination (噪声污染)** ——早期的失败路径、过时的中间观测一直挂在上下文里持续干扰后续决策。

核心方法: IterResearch 把长程研究重新形式化为 **MDP (马尔可夫决策过程) + 策略性工作区重建**。它不做线性累积, 而是维护一份 **evolving report (演进报告)** 作为记忆——每隔若干步就把已有信息**合成洞见**写进报告、丢弃冗余的原始内容, 然后基于"重建后的紧凑工作区"继续决策。这保证了在任意探索深度下推理容量保持一致 (不随步数增长而退化)。训练侧配套 **EAPO (Efficiency-Aware Policy Optimization)**: 用几何奖励折扣激励高效探索 (早找到答案的轨迹奖励更高) + **自适应下采样**稳定分布式训练。

关键结果: 6 个基准平均 **+14.5pp**; 把交互扩展到 **2048 次**时性能从 3.5% 飙升到 42.5%; 即使只作为纯 **prompting 策略** (不训练), 也能让 frontier 模型在长程任务上比 ReAct **+19.2pp**。

三维定位: 表示=evolving report; **更新**=周期性重构 (Markovian state reconstruction); **使用**=每轮把合成洞见**重置**进一个干净工作区。详见 deep-research/02-iterresearch.md。

意义：IterResearch 把"上下文该不该装下一切"这个问题给出了明确的否定回答——上下文是需要周期性 **curate** 的状态，不是无脑堆积的日志。它与 Harness-1 的"状态外置"是同一病症（mono-context 膨胀）的两条解法（policy 周期性重建 vs 环境侧维护），是 working-memory"更新侧"的标杆工作。

Memory Intelligence Agent (MIA) 📄 [786★ | 2026-04] — arXiv 2604.04503

问题动机：deep research agent (DRA) 把 LLM 推理与外部工具结合，但其记忆机制有两大顽疾——**记忆演化无效**（现有方法只会检索相似的过去轨迹，但记忆本身不会随使用变好）和**存储/检索成本**随时间上升（轨迹越积越多，检索越来越慢越贵）。

核心方法：MIA 提出 **Manager-Planner-Executor** 三组件架构：① **Memory Manager**（非参数记忆系统）——存储压缩后的历史搜索轨迹；② **Planner**（参数记忆 agent）——针对问题生成搜索计划，其能力以参数形式承载；③ **Executor**（执行器）——在搜索计划引导下实际检索与分析。训练用 **alternating RL**（交替强化学习）提升 Planner-Executor 协作。最关键的创新是 **bidirectional memory conversion loop**（参数↔非参数记忆双向转换循环）——非参数记忆（轨迹）可以蒸馏进参数记忆（Planner 能力），参数记忆也能外化为非参数记忆，二者循环转换实现高效记忆演化。Planner 还支持推理时 **on-the-fly** 持续演化（不中断推理过程），并用 reflection + unsupervised judgment 机制支持开放世界自演化。

关键结果：在 11 个基准上展示优越性（摘要未给具体数字，需读 PDF）。GitHub/项目热度 786★。

三维定位：表示=非参数（压缩轨迹）+ 参数（Planner）双重记忆；**更新**=参/非参双向转换 + 推理时 on-the-fly 演化；**使用**=运行时检索 + Planner 生成计划引导。

意义：MIA 是"记忆 = 运行时检索 + 双向演化"的代表（偏 DRA 方向，故非精读）。它把 GAM"运行时重构"的思想往前推了一步——不只是运行时检索，还让**参数与非参数记忆互相转化**，是记忆演化机制设计上的有趣探索。

Toward Generalist Autonomous Research via Hypothesis-Tree Refinement (Arbor) 📄 [Microsoft / RUC, 100↑ | 2026-06] — arXiv 2606.11926

问题动机：科学进步依赖"探索→实验→抽象"的反复循环——研究者测试候选方向、解读证据、把教训带进后续尝试。如何让 AI agent 在长程上自主跑完这个循环？现有自治科研多是"一连串孤立的局部尝试"，策略、执行、证据无法跨时间累积传递。

核心方法：Arbor 提出三件套：① **long-lived coordinator**（长生命周期协调器）——在整个研究周期内管理全局策略；② **short-lived executors**（短生命周期执行器）——在隔离的 **worktree** 中实现并测试单个假设；③ **Hypothesis Tree Refinement (HTR, 假设树精炼)**——一棵跨时间的持久树，把假设、产物、证据、蒸馏出的洞见全部链接起来。当执行结果返回时，Arbor 执行四个动作：**更新树、传**

播可复用教训、精炼搜索前沿、采纳已验证的改进。这把自治科研从"局部尝试序列"变成"策略/执行/证据跨时间累积的过程"。

关键结果：在 model training / harness engineering / data synthesis 三类**6 个真实研究任务**上，Arbor 在全部 6 个上拿到最佳 held-out 结果，平均相对 held-out 增益**超过 Codex 和 Claude Code 的 2.5 倍**（同任务接口、同资源预算）；在 **MLE-Bench Lite** 上用 GPT-5.5 达到 **86.36% Any Medal**，为其对比中最强。

三维定位：表示=Hypothesis Tree（跨时间的持久树，承载假设/产物/证据/洞见）；**更新**=结果返回时更新树 + 传播教训；**使用**=coordinator 基于树做全局策略决策。

意义：Arbor 把 working memory 推到 **tree 形式 + 跨时间传播**这一极（偏 auto-research，故非精读）。它与 Microsoft 系的 RHO（harness 自我演化）一道，代表"长程 agent 的记忆不只是上下文，而是一个可累积、可传播的结构化研究状态"——是 working-memory 在自治科研场景的高级形态。

B. 状态外化 & Harness 工程 (State Externalization & Harness)

把可恢复的"记账"从 policy 搬到环境侧的 harness / 文件系统，policy 只留语义决策。

Harness-1: Reinforcement Learning for Search Agents with State-Externalizing Harnesses ★ [Chroma / UIUC, 52↑ | 2026-06] — arXiv 2606.02373

问题动机：搜索 agent 通常被训练成"在不断增长的 transcript 上的 policy"——模型既要决定"怎么搜"，又要在脑子里**记住**"看过什么、哪些证据有用、哪些约束还没满足、哪些 claim 已被核实"。这是把太多例行的状态管理塞进了 **policy**。这些记账（bookkeeping）其实是可恢复的、环境能更可靠维护的，不该让 RL 去优化它——否则 RL 被迫同时学"语义搜索决策"和"可恢复的记账"两件事，样本效率低、泛化差。

核心方法：Harness-1 是一个 **20B 检索 subagent**，在一个 **stateful search harness**（有状态搜索 harness）内用 RL 训练。harness 在**环境侧**维护 working memory：**candidate pool**（候选池）、**importance-tagged curated set**（重要性标注的精选集）、**compact evidence links**（紧凑证据链接）、**verification records**（验证记录——哪些 claim 已核查）、压缩去重的 **observations**（观测）、**budget-aware context rendering**（预算感知的上下文渲染——按 token 预算决定给 policy 看什么）。policy 则只保留真正需要智能的**语义决策**：搜什么、留/弃哪些文档、验证什么、何时停。

关键结果：8 个检索基准（web / finance / patents / multi-hop QA）平均 curated recall **0.730**，比次优开源 subagent **+11.4pt**；在 **held-out transfer**（未见过的迁移基准）上增益尤强——说明"语义决策"比"记账"更可迁移（记账是任务特定的，语义能力是通用的）。GitHub 548★。

三维定位：表示·更新·使用全覆盖——表示=环境侧结构化工作记忆；更新=环境自动 bookkeeping（去重/压缩/重要性打标）；使用=budget-aware rendering。已成精读 [01-harness-1](#)。

意义：本专题核心论文。Harness-1 给出了一个可操作的设计原则——把"可恢复的状态"外置给环境，只让 RL 优化"不可恢复的语义决策"。这对所有长程 agent（不只搜索）都有借鉴意义，是"harness engineering"（系统级设计与权重训练同等重要）的精确实证，也是 working memory"表示侧离开 transcript"的标杆。

FS-Researcher: Test-Time Scaling for Long-Horizon Research Tasks with File-System-Based Agents ★ [52↑ | 2026-02] — arXiv 2602.01566

问题动机：长程研究任务（写一篇综述、做一次完整调研）会积累远超 context length 的信息，而且常需要多个 agent / 多个 session 协作——但上下文窗口装不下、也无法跨 session 共享。

核心方法：FS-Researcher 把文件系统当作持久外部记忆 + 跨 agent/session 的协调介质。两个角色：① **Context Builder**（上下文构建器）——像图书管理员一样，把检索到的信息写成结构化笔记、把原始源归档到一个分层知识库（hierarchical knowledge base），这个知识库的容量可以**远超 context length**；② **Report Writer**（报告撰写器）——基于这个文件系统知识库**逐节成文**。文件系统的好处：持久（session 间不丢）、可结构化组织、可被多个 agent 读写协调。

关键结果：在长程研究任务上通过"文件系统 test-time scaling"显著提升报告质量（具体数字见精读）。

三维定位：working-memory 表示=文件系统的代表作——把记忆从"上下文里的 token"外化成"磁盘上的分层知识库"。已成精读 [03-fs-researcher](#)。

意义：FS-Researcher 与 Harness-1 是"状态外化"的两种载体（文件系统 vs harness 内部结构），共同说明可恢复的研究状态应该放在 policy 之外的持久介质里。文件系统这一选择尤其朴素有力——它天然支持持久化、层级组织和多 agent 协调，是长程研究 agent 的实用记忆基座。

Retrospective Harness Optimization (RHO) 📄 [Microsoft 等, 52↑ | 2026-06] — arXiv 2606.05922（正式标题：*Evolving Agents in the Dark: Retrospective Harness Optimization via Self-Preference*）

问题动机：agent 的 harness（skills、tools、workflows、prompts）需要不断优化，但优化通常依赖带标注的验证集——这在很多真实场景不可得（没有 ground-truth 评分）。能不能只用过去的轨迹、完全自监督地优化 harness？

核心方法：RHO 是一个自监督 harness 优化方法，只用过去轨迹、无需外部评分。流程：① **coreset 选择**——从历史轨迹里挑出一个多样的、有挑战性的任务子集；② **并行重解**——重新跑这些选中的难任务；③ **自我分析**——agent 用 **self-validation**（自我验证）+ **self-consistency**（自我一致性）分析这

些 rollout；④ 候选生成与选择——生成多个 harness 更新候选，再用 **pairwise self-preference**（成对自我偏好）选出最好的。

关键结果：在软件工程 / 技术工作 / 知识工作三个领域评测；单轮优化就把 **SWE-Bench Pro** 通过率从 **59% 提到 78%**，且无需任何外部评分。分析显示 RHO 能有效针对过去的失败模式、在长程 session 中维持更高准确率。GitHub：github.com/wbopan/retro-harness。

三维定位：working-memory 之外的"**harness 自我演化**"代表（非精读）——它优化的是 agent 的"操作基座"而非"工作记忆"本身，但**完全靠过去轨迹自监督**这一点与 working memory 的"从历史中学"精神相通。

意义：RHO 把"agent 改进"从"需要人类标注的验证集"降到"只用自己跑过的轨迹 + 自我偏好"，是 agent democratization 的代表。它与 Arbor（同 Microsoft 系）一道说明，长程 agent 的"记忆"可以反过来驱动 harness 本身的进化。

C. 上下文管理：压缩 / mask / 剪枝 / 切换（Context Management）

上下文是需要主动调度的有限资源——删、缩、切换、编译成监督。

Masking Stale Observations Helps Search Agents — Until It Doesn't: A Regime Map and Its Mechanism ★ [McAuley-Lab, 62↑ | 2026-06] — arXiv 2606.00408

问题动机：长程 search agent 跨多次 tool call 积累大量检索内容，context 预算效率变得重要。一个最小干预是 **mask 掉过期观察（stale observations）**——把早期检索回来、现在已没用的页从上下文里移走。但何时这样做有用、为什么有用，此前并不清楚，工程上多凭直觉激进压缩。

核心方法：本文做了一次大规模系统实验——扫描 **4B-284B backbone × 3 个 retriever × 离线/实时 web 基准**。**核心发现：**masking 带来的准确率增益（相对"完全不做 context 管理"的准确率）呈现**非对称的倒 U 形 regime**——弱 retriever 下是平台（masking 没用，因为 retriever 本身召回差）、强 retriever 配中等模型时是**峰值**（masking 增益最大）、**模型饱和（很强）时骤降甚至变负**（强模型自己会忽略噪声，硬删反而误伤）。**机制解释：**masking 本质是一个 **token-for-turn 权衡**——移除模型已基本停止 attend 的旧观测，换取预算去做更多 turn；增益取决于 **retriever recall 与模型 implicit filtering capacity（隐式过滤能力）的交互**。

关键结果：把 context 管理重新定义为 **regime-dependent（依赖具体 regime）** 的干预——没有银弹。GitHub 16★。

三维定位：working-memory 使用侧"何时该删"的精读——而且给出了删的边界条件。已成精读 [02-masking](#)。

意义：这篇是对前面所有"激进 context 压缩"工作的重要补丁/反思——它用系统性实验证明 **context 管理策略必须匹配 (retriever recall × model filtering capacity)** 的具体 **regime**，不能无脑套用。这种"先别急着压缩，先搞清楚什么时候压缩才有用"的实证精神，对 working-memory 工程有直接的指导价值。

SWE-Pruner: Self-Adaptive Context Pruning for Coding Agents ★ [92↑ | 2026-01] — arXiv 2601.16746

问题动机：coding agent 在大代码库里工作时，把整个相关文件塞进上下文会导致几十万 token 的爆炸——既慢又贵。但人类程序员只读关键的 10–20 行就能定位 bug。如何让 agent 也"选择性 skim"？

核心方法：SWE-Pruner 模仿程序员的"选择性略读"。流程：① agent 先确定明确的**任务目标/intent**（如"关注错误处理"、"关注线程同步"）；② 引入一个**轻量 0.6B 神经 skimmer（略读器）**——按当前 intent 动态给每一行代码打相关度分，**只保留高分行**进入主 LLM 的上下文；③ skimmer 与主 agent 联合微调，让两者协调。

关键结果：在 agent 任务上实现 **23–54% 的 token 缩减**（成本相应下降），通过率几乎无损；单轮任务（LongCodeQA 类）最高 **14.84× 压缩**。

三维定位：working-memory 使用侧（**task-aware 剪枝**）的代表作——记忆不是删旧的（如 Masking），而是按当前任务目标只渲染相关的。已成精读 [05-swe-pruner](#)。

意义：SWE-Pruner 把 long-context coding 从"更大模型 + 更长上下文"的路线，推到"**小辅助模型按 intent 动态裁剪**"的路线。它与 Masking 互补——Masking 按"时效性"删（stale），SWE-Pruner 按"任务相关性"留（intent-aware）——共同说明上下文该按语义需求主动 curate，而非全量塞回。

QwenLong-L1.5: Post-Training Recipe for Long-Context Reasoning and Memory Management ★ [Tongyi, 113↑ | 2025-12] — arXiv 2512.12967

问题动机：即使把上下文窗口扩到很长，也装不下任意长的序列——总有超过窗口的场景。如何在"单遍长上下文推理"和"迭代式记忆处理"之间无缝切换，支撑超长（>4M token）输入？

核心方法：QwenLong-L1.5 是一套完整的长上下文后训练 **recipe**，三大件：① **长上下文数据合成 pipeline**——生成高质量长依赖训练数据；② **稳定化 RL (AEPO)**——长上下文 RL 的训练稳定性技术；③ **Memory-Augmented Architecture（记忆增强架构）**——承认"再长的窗口也装不下任意长序列"，用多阶段融合 RL 把单遍推理（single-pass）与迭代记忆处理（iterative memory processing）无缝结合，让模型在两种模式间按需切换，支撑 **>4M token**。

关键结果：在长上下文推理与记忆管理基准上达到 SOTA 级，支撑超 4M token 的输入处理（具体数字见精读）。

三维定位：表示·使用——表示触及 parametric（模型内化的长上下文能力）；使用=单遍 ⇌ 迭代记忆融合，按序列长度自适应切换。已成精读 [06-qwenlong-l15](#)。

意义：QwenLong-L1.5 代表了"长上下文不是靠无限扩窗口，而是靠单遍与记忆处理的智能切换"这一务实路线。它把"记忆管理"作为长上下文后训练的一等组件，是 working-memory 在"超长序列"极端场景下的工业级答案。

ACC: Compiling Agent Trajectories for Long-Context Training ★ [USTC, 59↑ | 2026-05] — arXiv 2605.21850

问题动机：agent 解题会产生海量轨迹，答案证据散落在多个 turn 里。但标准的 agent SFT 会 mask 掉 tool 响应（只对 agent 自己的输出算 loss），这造成了一个监督盲区——模型从没被显式训练过"如何跨多个 turn 整合分散的证据"。

核心方法：ACC（Agent trajectory Compilation for long-Context）把 search/SWE/DB agent 的轨迹编译成 long-context QA 训练样本——把原始问题 + 跨 turn 的 tool 响应/环境观测拼成一个长上下文输入，让模型显式学习跨段的长上下文推理。这相当于把"散落在轨迹各处的工作记忆"重新编译成可监督的训练信号。

关键结果：MRCR（多轮长上下文检索）+18.1、GraphWalks（图遍历推理）+7.6。

三维定位：working-memory **使用侧（把散落状态编译成监督）**的精读——它解决的是"如何训练模型用好分散在长上下文里的工作记忆"。已成精读 [04-acc](#)。

意义：ACC 揭示了一个被忽视的训练盲区——agent SFT 把 tool 响应 mask 掉，等于没教模型用工作记忆。把轨迹编译成 long-context QA 这一招，把"跨 turn 证据整合"从隐式期望变成显式监督，对所有需要长上下文工作记忆的 agent 训练都有借鉴价值。

Mindscape-Aware RAG (MiA-RAG) 📄 [115↑ | 2025-12] — arXiv 2512.17220

问题动机：人类理解长而复杂的文本时，靠的是对内容的整体语义表示（holistic semantic representation）——先建立一个全局的"这篇讲什么、结构如何"的认知骨架，再据此理解局部细节。但现有 RAG 系统缺乏这种全局指引——它们做局部 top-k 检索、局部生成，每次只看检索回来的几个片段，没有对整体内容的把握，因此在长上下文任务上表现挣扎（容易"只见树木不见森林"）。

核心方法：MiA-RAG 是首个把"mindscope-aware 检索与生成"形式化为统一 conditioning 范式的框架。它通过层次摘要（hierarchical summarization）构建一个 mindscope（全局语义表示）——对长文本逐层摘要出一个全局的语义骨架，然后让检索和生成都条件于这个全局视图：检索器据此形

成"信息更丰富的 query embedding" (带着全局语境去检索, 而非孤立匹配关键词), 生成器在"连贯的全局上下文"中对证据做推理 (每个局部判断都放在全局框架下)。即用一个全局 mindscape 同时引导"检索什么"和"如何生成"。

关键结果: 在长上下文任务上一致超过基线 (摘要只给定性结论"consistently surpasses baselines", 无具体数字, 需读 PDF)。8 作者。

三维定位: 长上下文"全局表示"的一例 (非精读, 偏 RAG) —— 它的"工作记忆"是一个全局 mindscape (语义骨架), 而非局部检索片段的堆叠。

意义: MiA-RAG 代表了 RAG 侧对"工作记忆"的一种独特理解——不是把检索片段堆进上下文, 而是先建一个全局语义骨架 (mindscape) 再 condition。这与本专题"上下文要主动组织、不要无脑堆积"的主线一致 (呼应 IterResearch 的 evolving report、Harness-1 的 curated set), 但 MiA-RAG 的组织方式是"自顶向下的全局摘要"而非"自底向上的证据 curate"。它提供了"全局摘要作为工作记忆锚点"的视角——当工作记忆很大时, 一个全局 mindscape 能帮 agent 保持"我在研究什么、整体结构如何"的把握, 避免在海量局部片段里迷失。这与 MindScape 类全局表示、以及 deep research 中的 evolving report 思路相通。

D. 记忆架构: latent / KV-cache / 稀疏注意力 (Memory Architectures)

把"记忆"做进模型内部——固定状态矩阵、KV-cache、稀疏注意力, 与序列长度解耦。

δ -mem: Efficient Online Memory for Large Language Models ★ [declare-lab, 128[↑] | 2026-05] — arXiv 2605.12357

问题动机: 给 LLM 加长程记忆, 常见做法要么 full fine-tune (贵)、要么换 backbone 架构 (破坏已有能力)、要么扩上下文 (治标不治本)。能不能给一个冻结的 full-attention backbone 加一个轻量、在线、不动权重的记忆模块?

核心方法: δ -mem 给冻结 backbone 加一个紧凑的在线关联记忆 (associative memory): 用 delta-rule (增量规则) 把历史信息压进一个 8×8 的固定大小状态矩阵 (与序列长度解耦——历史再长, 矩阵大小不变), 记忆 readout 时对注意力做低秩修正 (low-rank correction)。整个过程无需 full fine-tuning、无需换 backbone、无需扩上下文——backbone 冻结, 只训练这个小记忆模块。

关键结果: MemoryAgentBench 上达到 **1.31x** 的提升 (相对基线)。

三维定位: 表示·更新·使用全覆盖——表示= 8×8 固定状态矩阵 (latent 形式); 更新=delta-rule 在线更新; 使用=对注意力做低秩修正注入。已成精读 [07-delta-mem](#)。

意义： δ -mem 是 working memory **latent 形式 + 在线更新**的代表——它证明了"给冻结模型外挂一个固定大小的可更新状态矩阵"是一条轻量、不破坏原模型的记忆增强路线。这与 token-level 工作记忆 (Harness-1) 形成鲜明对比：一个把记忆做进模型内部的 latent 状态，一个把记忆放在上下文/环境的 token 里。

MSA: Memory Sparse Attention for Efficient End-to-End Memory Model Scaling to 100M Tokens 📄 [EverMind, 3469★ | 2026-03] — arXiv 2603.23516

问题动机：full-attention LLM 通常被限制在 ~1M token。现有扩展方法（混合线性注意力、RNN 式固定记忆、RAG/agent）各有短板——精度损失、延迟上升、或缺乏端到端优化。如何做一个**端到端可训练、能扩到亿级 token**的记忆模型？

核心方法：MSA 是一个**端到端可训练、高效、可大规模扩展**的记忆模型框架。核心创新：① **scalable sparse attention**（可扩展稀疏注意力）+ **document-wise RoPE**——实现训练与推理的线性复杂度；② **KV cache 压缩 + Memory Parallel**——让 **2xA800 GPU 就能做 100M token 推理**；③ **Memory Interleaving**（记忆交织）——支持"跨分散记忆段的复杂多跳推理"。

关键结果：从 **16K 扩展到 100M token**，性能退化 **<9%**；在长上下文基准上超过 frontier LLM、SOTA RAG 系统和领先的记忆 agent。项目热度 3469★。

三维定位：working memory 的**架构级极致扩展**（非精读，偏底层架构）——它的"记忆"是端到端训练的稀疏注意力 + KV 压缩，把序列长度推到亿级。

意义：MSA 代表了"把记忆做进模型架构、端到端扩到亿级 token"这条最激进的路线。它与 δ -mem（固定状态矩阵）同属"记忆架构"派，但走向相反——MSA 追求**极致规模**（100M token）， δ -mem 追求**极致轻量**（8x8 矩阵 + 冻结 backbone），共同勾勒出"记忆架构"设计的两端。

HERMES: KV Cache as Hierarchical Memory for Efficient Streaming Video Understanding 📄 [OpenMOSS, 75↑ | 2026-01, ACL 2026 Main] — arXiv 2601.14724

问题动机：流式视频理解（边播放边回答问题）要求**实时响应**，但每来一个 query 都重新计算整段视频的注意力，开销巨大——视频越长、query 越多，重复计算越多，无法满足实时性。能否**复用已经算好的 KV-cache** 来避免重复计算？

核心方法：HERMES 是一个 **training-free 架构**（无需训练，即插即用），把 **KV-cache 重新概念化**为**分层记忆框架（hierarchical memory）**——核心洞察是 KV-cache 天然编码了**多个粒度**的视频信息（细粒度的帧级、粗粒度的片段级），HERMES 把它组织成**分层记忆**并复用：当新 query 到来时，直接在已有的多粒度 KV-cache 上做注意力，**无需重新编码整段视频**即可实时响应。这相当于把"已经算好的历史计算结果"当作一种现成的、可复用的工作记忆。

关键结果：TTFT（首 token 时间）比此前 SOTA 快 **10x**；相对均匀采样 token 减少最高 **68%**；流式准确率提升最高 **11.4%**。ACL 2026 Main 收录。

三维定位：working memory 的 **KV-cache** 形式代表（非精读，偏视频）——它把已经算好的 KV-cache 当作分层工作记忆复用，是"表示"维度的一种独特形态（记忆=已计算的 KV-cache 而非 token/latent）。

意义：HERMES 提供了一个优雅的视角——**KV-cache 本身就是一种现成的、多粒度的工作记忆**，training-free 地复用它就能大幅提速（TTFT 10x）。这对"如何在重算的前提下利用历史计算结果"这一通用问题有启发：working memory 不一定要新建数据结构（如 Harness-1 的 candidate pool）或新训模型（如 δ -mem 的状态矩阵），有时**已有的 KV-cache 就是免费的分层记忆**。它与 δ -mem（固定状态矩阵）、MSA（稀疏注意力扩到亿级 token）同属本专题"记忆架构"派，但 HERMES 最轻量——不改架构、不训练，纯复用现成 KV-cache，是"KV-cache 即记忆"思路的代表。

Latent Collaboration in Multi-Agent Systems (LatentMAS) 📄 [Gen-Verse, 127↑ | 2025-11, ICML 2026 Spotlight] — arXiv 2511.20639

问题动机：多 agent 系统（MAS）把 LLM 从单模型推理扩展到系统级协作，但传统 MAS 的 agent 间通信靠**文本中介**——每次都要把内部状态 encode 成文本再传，造成信息损失和额外开销。

核心方法：LatentMAS 是端到端 **training-free** 的框架，让 agent **直接在连续潜空间协作**。两个机制：
① **latent thought generation（潜思生成）**——每个 agent 通过**最后一层 hidden embedding** 做自回归推理，而非生成文本 token；
② **shared latent working memory（共享潜工作记忆）**——保存并传递每个 agent 的内部表示和潜思，实现**"无损信息交换、无需重新编码"**。论文提供理论分析，论证其比文本 MAS 有更高表达力、无损信息保留、更低复杂度。

关键结果：9 个基准（数学/科学推理、常识理解、代码生成）上准确率最高 **+14.6%**；输出 token 减少 **70.8%–83.7%**；端到端推理快 **4x–4.3x**。GitHub：github.com/Gen-Verse/LatentMAS。

三维定位：working memory 的 **latent** 形式一例（非精读，偏 MAS）——它的"工作记忆"是 agent 间共享的潜空间表示。

意义：LatentMAS 把多 agent 通信从"文本中介"搬到"潜空间直传"，提出 **latent working memory** 概念。这与 Recursive MAS（huggingface/16）的 latent communication 同源，共同代表"multi-agent 协作的信息载体可以是 hidden state 而非 text"这一前沿方向，是 working memory 在 MAS 场景的 latent 化探索。

E. 可学习 / 可演化的记忆（Learnable & Evolving Memory）

"记什么、怎么记、怎么巩固"不再硬编码，而是可学习、可演化、可离线巩固。

MemSkill: Learning and Evolving Memory Skills for Self-Evolving Agents ★ [NTU, 63↑ | 2026-02] — arXiv 2602.02474

问题动机：agent 的记忆操作（提取关键信息、巩固记忆、剪枝冗余）通常是**硬编码的固定流程**——但"什么该提取、怎么巩固、何时剪枝"其实高度任务依赖，固定流程难以适应多样场景。

核心方法：MemSkill 把"提取/巩固/剪枝记忆"的固定操作**重构为可学习、可演化的 memory skills**（记忆技能）。三角色：① **controller**（控制器）——选用哪个 memory skill；② **executor**（执行器）——用选中的 skill 产生记忆；③ **designer**（设计器）——复盘难例、演化 skill 集（增删改 skill）。这样"怎么记"就从人类先验变成了可学习、可随经验演化的 skill 库。

关键结果：在 self-evolving agent 场景下，可学习的 memory skill 优于固定记忆操作（具体数字见精读）。

三维定位：working-memory **更新策略可学化**的代表——"记什么、怎么记"被做成可学习、可演化的 skill。已成精读 [11-memskill-memento](#)（与 Memento-Skills 合并）。

意义：MemSkill 把"记忆操作"从硬编码提升为"可学习的 skill"，是 working memory"更新侧"从人工先验走向自动演化的代表。它与 TaskMem（RL 学 memorization policy）思路相通——都在回答"该记什么不该由人拍脑袋，而该由 agent 从任务中学"。

Memento-Skills: Let Agents Design Agents ★ [UCL, 58↑ | 2026-03] — arXiv 2603.18743

问题动机：如何让 agent 在**不更新 LLM 参数**的前提下，从经验中持续学习、自我改进？

核心方法：Memento-Skills 用 **memory-based RL + stateful prompts**：可复用的 skill（结构化 markdown 文件）充当**持久、演化的记忆**；采用 **Read-Write 反思式学习**——**读阶段** skill router 选用相关 skill，**写阶段**更新/扩充 skill 库。整个过程**不更新 LLM 参数**，全靠外化的 skill/prompt 演化实现 agent 自改进。

关键结果：在多任务上通过 skill 库的读写演化实现持续改进（具体数字见精读）。

三维定位：working-memory **更新（策略可学）**——记忆即可读写演化的 markdown skill。已成精读 [11-memskill-memento](#)（与 MemSkill 合并）。

意义：Memento-Skills 代表"**记忆 = 可演化的 skill 文件 + 不动权重**"这一 frozen-model 时代的实用路线。它与 MemSkill 合并精读，共同说明记忆操作可以完全外化为 markdown skill 的读写，是 working memory 与 skill engineering 的交汇点。

EvoArena: Tracking Memory Evolution for Robust LLM Agents in Dynamic Environments ★ [MIT, 90↑ | 2026-06] — arXiv 2606.13681

问题动机：现有 agent 评测大多假设**静态环境**——但真实部署中环境会**渐进变化**（软件更新、社交规则演变、终端状态迁移）。agent 的记忆必须能**记录并适应这种环境演化**，否则会用过时记忆做错误决策。

核心方法：EvoArena 把环境变化建模为 terminal/software/social 三类场景上的**渐进 update 序列**（一系列对环境结构化改动）。配套 **EvoMem**——一种 **patch-based 记忆（补丁式记忆）**，以**结构化 update history（更新历史）** 记录记忆的演化（每次环境变化作为一个 patch 追加），让 agent 始终知道“环境现在是什么样、之前怎么变过来的”。

关键结果：当前 agent 在动态环境上仅 **39.6%**；EvoMem 在 GAIA/LoCoMo 上分别 **+6.1%/+4.8%**，chain-level（跨多步一致性）**+3.7%**。

三维定位：working-memory **更新（记录演化）** + 评测的精读——它既提出了“记录环境演化”的 patch 式更新机制，又建立了动态环境评测基准。已成精读 [08-evoarena-evomem](#)。

意义：EvoArena 指出了被忽视的评测盲区——**真实环境是动态的，记忆必须记录演化**。它把 working memory 的研究从“静态环境下管理上下文”推进到“动态环境下追踪并适应变化”，是记忆“更新侧”和“评测侧”的双重贡献。

Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories ★ [Google, 29↑ | 2026-06] — arXiv 2606.03979

问题动机：人类靠睡眠把脆弱的短期记忆巩固成稳定的长期记忆。LLM 能不能也有一个“睡眠”阶段，把交互中获得的**短期记忆离线蒸馏进稳定的长期参数**？

核心方法：提出 **Sleep 范式**：把脆弱的**短期记忆经 replay（回放）蒸馏进稳定的长期参数**——其中 **Knowledge Seeding（知识播种）** 是一种“小自我→大网络”的向上蒸馏（用模型自己产生的小规模知识种子去更新大网络），并用 **Dreaming（做梦）**——RL 自生成课程——来自我精进。整个过程是 agent 在“清醒”交互之外的“离线巩固”阶段。

关键结果：作为 proof-of-concept，展示了短期记忆向长期参数巩固的可行性（具体数字见精读/PDF）。

三维定位：working-memory 与 **long-term 参数化记忆的桥接（更新侧·离线巩固）** 精读——它处理的是“工作记忆如何沉淀为长期参数”。已成精读见 [10-externalization-efficiency](#) 关联讨论与本条。

意义：Sleep 范式触及一个深刻问题——**working memory（短期、token/上下文）与 long-term memory（参数）之间的巩固边界**。它把“睡眠巩固”这一生物学机制引入 LLM，提出离线把短期记忆蒸馏进权重的路线，是 working memory 与参数记忆桥接的前沿探索（虽仍是 proof-of-concept）。

Task-Focused Memorization for Multimodal Agents (TaskMem) 📄 [ByteDance-Seed, 38↑ | 2026-05] — arXiv 2605.31075

问题动机：多模态 agent（如具身 agent）持续感知、推理、行动，接收无界的多模态观测流。从这种"信息的组合爆炸"中，agent 必须**选择性保留**与其角色相关、对未来任务有价值的内容——核心挑战是"该记什么"，而非记忆模块怎么设计。

核心方法：TaskMem 把记忆生成重构为可学习的 **memorization policy**（记忆策略），是一个 **RL 框架**，让策略能**动态调整关注点**以匹配真实任务需求。两阶段训练：① **Phase One（学怎么记）**——在基本保真度要求下优化记忆质量；② **Phase Two（学该记什么，部署后）**——在 base MLLM 上 tune 一个 adapter，用**近期环境任务定义 reward model**，引导记忆策略偏向**任务相关内容**。评测时把 VideoMME/EgoLife/EgoTempo 重构成**流式基准**（观测顺序到达、任务在线到来），且问题只能用 **agent 的记忆回答、不能看原始视频**，以隔离记忆评估。

关键结果：基于 Qwen3-VL-30B-A3B，TaskMem 在三个基准上 VQA 准确率分别 **+6.3% / +7.0% / +5.3%**。

三维定位：working-memory **更新策略可学化**（多模态侧，非精读）——用两阶段 RL 学"怎么记"与"该记什么"。

意义：TaskMem 把"该记什么"这一 working memory 的核心难题，明确建模成**可学习的 memorization policy**，并用"部署后用近期任务定义 reward"实现任务自适应。它与 MemSkill 共同代表"记忆的写入策略应由 RL 学习"这一方向，并把它扩展到了多模态/具身场景。

RubricEM: Meta-RL with Rubric-guided Policy Decomposition ★ [Google, 79↑ | 2026-05] — arXiv 2605.10899

问题动机：训练 deep research agent 把 RL 推到"可验证奖励之外"的领域——它们的输出（长报告）没有 ground-truth、轨迹跨很多工具决策、标准后训练几乎没机制把"过去的尝试"变成"可复用的经验"。

核心方法：RubricEM 主张 **rubric** 不应只当最终答案的评判器，而应充当统一接口——同时结构化 **policy 执行、judge 反馈、和 agent memory** 三者。框架三组件：① **stagewise policy decomposition**（分阶段策略分解）——把 planning/evidence gathering/review/synthesis 各阶段都 condition 在自生成的 rubric 上，使轨迹 stage-aware；② **Stage-Structured GRPO**——用分阶段 rubric 判断分配 credit，为长程优化提供更密的语义反馈；③ **reflection meta-policy**（反思元策略）——一个共享 backbone 的策略，把判过的轨迹蒸馏成"可复用的 rubric-grounded 指导"供未来尝试使用。

关键结果：RubricEM-8B 在 4 个长报告研究基准上超过同级开源模型、逼近商用 deep-research 系统（摘要未给具体数字）。63 页长文。

三维定位："**rubric** 即记忆接口"的一例（非精读，偏 RL）——**rubric** 在这里同时是执行接口、评判反馈和 agent memory。

意义：RubricEM 提出了一个统一视角——**rubric** 可以三位一体地充当执行/评判/记忆接口。它的 reflection meta-policy"把判过的轨迹蒸成可复用指导"，本质是一种 experiential working memory。这与 DR Tulu 的演化 rubric（deep-research/03）呼应，把 rubric 从"评分工具"提升为"记忆与经验的载体"。

F. 综述 & 统一框架（Surveys & Frameworks）

Memory in the Age of AI Agents ★ [157↑ | 2025-12] — arXiv 2512.13564

问题动机：agent memory 研究快速膨胀、关注度空前，但领域日益碎片化——各种工作对"记忆"的定义、分类、机制各说各话，缺乏统一语言。

核心方法：这是 agent memory 的最新权威综述。它明确把 agent memory 与 LLM memory / RAG / context engineering 区分开（agent memory 强调跨交互的状态承载，而非单纯的知识库或上下文工程）。核心贡献是用 三透镜统一框架梳理全领域：**forms**（形式：token-level / parametric / latent）× **functions**（功能：factual / experiential / working）× **dynamics**（动态：formed / evolved / retrieved）。任何一篇记忆工作都能在这个 3×3×3 的坐标里定位。

关键结果：综合数百篇工作，提供术语体系 + 分类框架 + 开放挑战。

三维定位：三维框架本身——本专题"表示/更新/使用"三维度地图正是对这套 forms×functions×dynamics 框架的本地化应用。已成精读 09-memory-survey。

意义：本专题的概念锚点与阅读地图。它给整个 agent memory 领域提供了统一语言——working memory 在这个框架里是"功能上偏 working、形式偏 token-level、动态最活跃（formed→evolved→retrieved 全活跃）"的一支。本专题聚焦的正是这一支。

Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering ★ [SJTU, 52↑ | 2026-04] — arXiv 2604.08224

问题动机：agent 基础设施（记忆、skill、协议、harness）各自为政地发展，缺乏统一理论解释它们之间的关系。

核心方法：本文用 externalization（外化）作为统一透镜回顾 agent 基础设施——memory 把状态跨时间外化、skills 把过程外化、protocols 把交互外化、harness 是协调它们的统一层。并提出一条历史演进主线：从 weights（权重）→ context（上下文）→ harness（基座）——agent 能力的承载正在从"训进权重"转向"放进上下文"再转向"放进 harness"。

关键结果：提供 externalization 统一框架 + 历史演进视角。

三维定位：表示·更新（理论）——它把 working memory 的"状态外化"放进了更大的"一切皆外化"框架里。已成精读 [10-externalization-efficiency](#)（与 Toward Efficient Agents 合并）。

意义：本专题的理论框架。它把 Harness-1（状态外化）、FS-Researcher（文件系统外化）、Memento-Skills（skill 外化）等本专题工作统一在"externalization"这一概念下，并给出"weights→context→harness"的演进史观——精确解释了为什么 working memory 正在"离开权重、离开 transcript、走向环境侧 harness"。

Toward Efficient Agents: Memory, Tool learning, and Planning ★（合并） [Shanghai AI Lab, 57↑ | 2026-01] — arXiv 2601.14192

问题动机：agent 系统越来越强，但效率（token、延迟、成本）成为落地瓶颈。如何从 memory/tool/planning 三组件系统审视 agent 效率？

核心方法：本文从 **memory / tool learning / planning** 三组件审视 agent 效率。在记忆侧的核心共识是：**bounding context via compression and management**（用压缩与管理来界定上下文）——即上下文不能无限膨胀，必须主动压缩和管理才能保持效率。

关键结果：提供 agent 效率的 Pareto 视角与三组件分析。

三维定位：表示·更新（理论）——它把 working memory 的"上下文压缩与管理"放进 agent 效率的大图里。与 [10-externalization-efficiency](#) 合并精读（效率/外化双视角）。

意义：这篇从"效率"角度为本专题提供了动机支撑——**working memory 的压缩/管理不只是为了能力，更是为了效率（token/延迟/成本）**。它与 Externalization Review 合并精读，共同构成本专题的理论双视角：一个讲"外化"（状态去哪），一个讲"效率"（为什么要压缩管理）。

G. 评测 & 训练数据（Benchmarks & Data）

HaluMem: Evaluating Hallucinations in Memory Systems of Agents 📄 [95↑ | 2025-11] — arXiv 2511.03506

问题动机：记忆系统让 LLM/agent 实现长期学习和持续交互，但在记忆存储和检索时频繁出现**记忆幻觉**——fabrication（编造）、error（错误）、conflict（冲突）、omission（遗漏）。现有评测多是端到端 QA，难以定位幻觉具体发生在记忆系统的哪个操作阶段。

核心方法：HaluMem 是首个面向记忆系统的操作级（operation-level）幻觉评测基准。它定义三个评测任务——memory extraction（记忆提取）、memory updating（记忆更新）、memory question answering（记忆问答）——以揭示不同操作阶段的幻觉行为。配套构建 user-centric 多轮人机交互数

数据集 **HaluMem-Medium** 和 **HaluMem-Long**，各含约 **15k 记忆点 + 3.5k 多类型问题**，单用户平均对话长度达 **1.5k / 2.6k 轮**、上下文超 **1M token**。

关键结果：实证发现现有记忆系统在**提取和更新阶段**就会产生并累积幻觉，随后传播到问答阶段。建议未来研究可解释、受约束的记忆操作机制来系统性抑制幻觉。

三维定位：记忆**可靠性评测**（非精读）——它把记忆质量评测从"最终答案对错"细化到"哪个操作阶段开始出幻觉"。

意义：HaluMem 把记忆评测从"端到端 QA"推进到"操作级幻觉定位"，揭示了**幻觉主要在提取/更新阶段产生并向下传播**这一关键洞察。这对 working memory 的"更新侧"设计有直接警示——写入记忆时的幻觉会污染整条下游，呼应了 EvoMem、MemSkill 等对"受约束、可追溯的记忆更新"的追求。

LMEB: Long-horizon Memory Embedding Benchmark 📄 [73↑ | 2026-03] — arXiv 2603.12572

问题动机：现有文本 embedding 基准（如 MTEB）聚焦传统的段落检索（给一个 query 找最相关的文档段），不评估对"**碎片化（fragmented）**、**上下文依赖（context-dependent）**、**时间上遥远**

（temporally distant）"信息的处理——而这正是长程记忆检索的核心。working memory 的"使用侧"恰恰需要：从一大团跨多轮、多时间点积累的工作记忆里，检索出当前推理需要的那些**分散、时间遥远、依赖上下文才能理解的片段**——这与"找最相似段落"是本质不同的能力。

核心方法：LMEB 是长程记忆**检索/embedding 评测基准**，跨**四种记忆类型**——**episodic**（情景，特定事件）/ **dialogue**（对话）/ **semantic**（语义，更高抽象）/ **procedural**（程序/任务流程）——这些类型在**抽象层级和时间依赖**上各不相同（episodic 强时间依赖、semantic 强抽象）。规模：**22 个数据集**、**193 个零样本检索任务**、**15 个 embedding 模型**（从数亿到百亿参数），覆盖广泛的长程记忆检索场景。

关键结果：① LMEB 提供"合理的难度"（不饱和、有区分度）；② **更大的模型不总是更好**——参数规模不直接等于长程记忆检索能力；③ **LMEB 与 MTEB 测的是正交能力**——传统段落检索强不代表长程记忆检索强。项目页：kalm-embedding.github.io/LMEB.github.io。

三维定位：记忆**使用侧（检索）评测**（非精读）——它评估的是"如何从长程记忆里检索出碎片化、时间遥远的信息"，是 working memory"怎么用"维度的专门评测。

意义：LMEB 揭示了一个对本专题至关重要的事实——**长程记忆检索是一种独立于传统段落检索的能力**（与 MTEB 正交）。这对 working memory 的"使用侧"有直接指导意义：当 Harness-1、GAM 等系统需要从环境侧 working memory / page-store 里检索分散、时间遥远的片段时，**不能直接套用通用检索模型（MTEB 强的）**，而需要专门优化的长程记忆 embedding。它和 HaluMem（记忆操作级幻觉评测）、

EvoArena（动态环境记忆评测）共同构成本专题"记忆评测"的多维度——分别覆盖检索能力、幻觉可靠性、动态适应性，说明"最终任务对"已不足以衡量记忆系统的各个侧面。

daVinci-Agency: Unlocking Long-Horizon Agency Data-Efficiently 📄 [GAIR, 53↑ | 2026-02] — arXiv 2602.02619

问题动机：LLM 在短任务上表现好，但在长程 agentic workflows 上挣扎，瓶颈是**数据稀缺**——现有合成方法要么局限于单一特征场景，要么需要昂贵的人工标注。

核心方法：daVinci-Agency 通过**真实世界软件演化**重新框定数据合成，论证 **Pull Request (PR) 序列天然蕴含长程学习的监督信号**。它从 chain-of-PRs 中挖掘监督，三个互锁机制：① **渐进任务分解**（通过连续 commit）；② **长期一致性强制**（通过统一的功能目标）；③ **可验证精炼**（来自真实的 bug-fix 轨迹）。相比合成替代方案，**PR-grounded 结构天然保留了因果依赖和迭代精炼**。

关键结果：每条轨迹平均 ~85k token、**116 次 tool call**；仅用 **239 个训练样本** fine-tune GLM-4.6，就在 **Toolathlon 上获得 47% 相对增益**——数据效率极高。

三维定位：偏"长程 agent 训练数据合成"（非精读），与 working memory 的**"跨阶段一致性"**相关——PR 序列天然编码了长程任务里"前后阶段如何一致演进"的结构。

意义：daVinci-Agency 提供了一个巧妙的洞察——**真实 PR 序列天然就是长程监督信号**，且因果依赖/迭代精炼都被保留。它与 working memory 的关联在于：训练长程 agent 用好工作记忆，需要"跨阶段一致性"的监督数据，而 PR 链恰好天然提供了这种结构，且数据效率惊人（239 样本）。

三、精读论文清单（11 篇 / 9 个文件）

文件	论文	月/票	三维主贡献	一句话
01-harness-1	Harness-1	06 / 52↑	表示·更新·使用（全）	环境侧 working memory, policy 只留语义决策
02-masking	Masking Stale Observations	06 / 62↑	使用（何时删）	mask 增益呈 regime-dependent 倒 U
03-fs-researcher	FS-Researcher	02 / 52↑	表示（文件系统）	文件系统当持久外部记忆 + 协调介质
04-acc	ACC	05 / 59↑	使用（编译成监督）	把散落 turn 编译成 long-context 训练信号

文件	论文	月/票	三维主贡献	一句话
05-swe-pruner	SWE-Pruner	01 / 92↑	使用（剪枝）	task-aware 神经 skimmer 自适应剪枝
06-qwenlong-l15	QwenLong-L1.5	12 / 113↑	表示·使用	单遍 $\Rightarrow$ 迭代记忆融合，撑 >4M token
07-delta-mem	δ -mem	05 / 128↑	表示·更新·使用	8×8 在线状态矩阵 + delta 更新 + 低秩注入
08-evoarena-evomem	EvoArena / EvoMem	06 / 90↑	更新（记录演化）	patch-based 记忆记录环境演化 + 评测
09-memory-survey	Memory in the Age of AI Agents	12 / 157↑	三维框架本身	forms×functions×dynamics 统一透镜
10-externalization-efficiency	Externalization Review + Efficient Agents	04·01 / 52↑·57↑	表示·更新（理论）	externalization 框架 + 效率 Pareto
11-memskill-memento	MemSkill + Memento-Skills	02·03 / 63↑·58↑	更新（策略可学）	记忆操作 = 可学习/可演化的 skill

去重交叉引用（已在 [deep-research/](#) 精读）：

- **GAM**（[deep-research/11](#)）——JIT 记忆，运行时按需检索整合。
- **IterResearch**（[deep-research/02](#)）——Markovian 工作区重构，演进报告作记忆。

四、2025-11 → 2026-06 关键趋势速览

围绕 **表示 / 更新 / 使用** 三维度提炼。

1. **【表示】working memory 正在“离开 transcript”**：从单一膨胀上下文，转向环境侧结构化状态（Harness-1 的 candidate pool/evidence links）、文件系统（FS-Researcher）、演进报告（IterResearch）、持久树（Arbor）、固定状态矩阵（ δ -mem）。共识——可恢复的状态不该塞在 **policy** 里。

2. 【更新】"谁来记、何时记"成为设计核心：环境自动 bookkeeping (Harness-1)、周期性重构 (IterResearch)、在线 delta-rule (δ -mem)、patch 演化历史 (EvoMem)、**RL 学到的 memorization policy** (TaskMem/MemSkill)、**离线 Sleep 巩固** (LMs Need Sleep)。趋势——把"记什么/怎么记"从硬编码人类先验，变成可学习、可演化、可离线巩固的过程。
3. 【使用】上下文从"全量塞回"变成"主动调度的资源"：budget-aware rendering (Harness-1)、mask stale 观测 (Masking, 但有 **regime 边界**)、task-aware 剪枝 (SWE-Pruner)、运行时 deep-research 式检索 (GAM/MIA)、低秩注入注意力 (δ -mem)、把散落 turn 编译成监督 (ACC)。关键发现——**没有银弹**：Masking 证明 context 删减的收益强烈依赖 retriever 召回 $\times$ 模型隐式过滤容量的交互。
4. 【RL 视角】把记账从 **policy** 剥离，**RL 才学得动**：Harness-1 与多篇共同主张——强化学习不该既学语义决策又学可恢复记账；**state externalization 让 held-out transfer 增益尤强**。
5. 【评测视角】从"静态环境"走向"演化环境"：EvoArena 指出真实部署是动态的，记忆必须**记录环境演化**；HaluMem 关注记忆**幻觉** (fabrication/conflict/omission)；LMEB 评长程记忆检索——"最终任务对"已不足以衡量记忆系统。
6. 【统一框架成型】**externalization 与 forms $\times$ functions $\times$ dynamics**：[09-memory-survey](#) 与 [10-externalization-efficiency](#) 给出两套互补的统一语言——**功能上区分 factual/experiential/working**，**形式上区分 token/parametric/latent**，**动态上区分 formed/evolved/retrieved**；working memory 是其中"形式偏 token、动态最活跃"的一支。

仍待解决：① working memory 的**最优粒度**（行级？观测级？语义块级？）缺乏统一结论；② **更新的稳定性**——在线 delta / patch / RL policy 在超长历史下是否漂移；③ 使用的 **regime 依赖** (Masking) 尚无自适应开关；④ working memory 与 long-term 参数记忆的**巩固边界** (Sleep) 仍是 proof-of-concept；⑤ **评测**仍多用 LLM-judge，记忆幻觉的客观度量未统一。

五、数据来源说明

- 论文元数据来自 **HuggingFace Papers 月榜** (huggingface.co/papers/month/YYYY-MM), 2025-11 ~ 2026-06 共 8 个月、每月 top50。
- 笔记内容基于 **arXiv 摘要**（部分含 GitHub README / 全文）撰写；模型/数据规模数字以论文摘要为准，摘要未披露处统一标注"需读 PDF"。
- 与 deep-research/、evolve/、huggingface/ 三个既有专题**去重**：重叠论文仅交叉引用、不重复精读。